Avaliação Acadêmica: Algoritmos e Programação em Python

Curso: Área de Tecnologia da Informação (TI) – Unilasalle

Professor: M.e Rafael Gastão

Formato: Padrão ENADE (Múltipla Escolha com Itens Analíticos)

Questão 1: Fundamentos de Algoritmos e Composição de Software (Aula 1)

Contexto: Uma solução digital de software, de acordo com as diretrizes metodológicas do Unilasalle, baseia-se na composição integrada e na interação contínua entre dois pilares estruturais básicos: **Processos** e **Dados**.

Enunciado: A partir desse princípio arquitetural, analise as afirmativas a seguir:

I. Os processos (programas) definem as ações lógicas e comportamentais do sistema, manipulando e transformando os dados recebidos.

II. Os dados (arquivos ou bancos de dados) representam o estado e a informação persistida ou temporária necessária à execução das funcionalidades.

III. De acordo com a taxonomia relacional, dados atômicos são classificados como conjuntos heterogêneos capazes de armazenar múltiplos tipos de dados simultaneamente sob uma única estrutura.

É correto o que se afirmar em:

A) I, apenas.

B) III, apenas.

C) I e II, apenas.

D) II e III, apenas.

E) I, II e III.

- **Gabarito:** C
- **Justificativa:** As afirmativas I e II estão corretas porque refletem com exatidão a divisão funcional de uma solução de software (Processos + Dados) e sua contínua troca de informações. A afirmativa III está incorreta porque dados atômicos representam um único valor temporal discreto (ex: um valor primitivo isolado como um float), ao passo que o agrupamento de múltiplos tipos distintos sob um mesmo nome qualifica os dados heterogêneos multivalorados (registros/estruturas).

Questão 2: Sequenciamento Lógico e Inversão de Instruções (Aula 1)

Contexto: No desenvolvimento de algoritmos lineares expressos em Português Estruturado (Linguagem Natural), a ordem das instruções dita a correção do processamento. Considere o algoritmo básico de cálculo de médias apresentado em aula:

```
Programa Calcula_Media

Var

  P1, P2, Media : real

Inicio

  Leia (P1, P2)

  Media <- (P1 + P2) / 2

  Escreva (“Media Final: ”, Media)

Fim
```

Enunciado: Caso um estudante modifique a estrutura do algoritmo invertendo a posição da linha do comando de leitura (Leia (P1, P2)) com a linha da instrução de atribuição aritmética (Media <- (P1 + P2) / 2), qual será o impacto lógico imediato sobre a execução do sistema?

A) O algoritmo executará corretamente, dado que o bloco Var pré-inicializa todas as variáveis reais com zero automaticamente.

B) Ocorrerá um erro de sintaxe estrutural que impede o interpretador de traduzir o bloco Inicio.

C) O programa apresentará uma falha crítica de lógica ou indeterminação, pois calculará a variável Media a partir de valores indefinidos (ou lixo de memória) contidos em P1 e P2 antes que o usuário digite as notas.

D) O comando Escreva ajustará o fluxo dinamicamente, solicitando a entrada de dados retroativa ao usuário.

E) O algoritmo converterá os tipos reais em dados multivalorados homogêneos dinamicamente devido à inversão.

- **Gabarito:** C
- **Justificativa:** Os algoritmos sequenciais seguem um fluxo estrito de execução passo a passo. Efetuar o cálculo aritmético da média utilizando variáveis que ainda não passaram pelo processo de leitura e recepção de dados externos gera um erro de lógica grave, pois o computador operará com valores ausentes ou indeterminados no instante da atribuição.

Questão 3: Tomada de Decisão Lógica e Condicionais Aninhadas (Aula 3)

Contexto: As estruturas condicionais (comandos de decisão) fornecem flexibilidade aos algoritmos ao desviarem o fluxo de execução do programa baseado em condições lógicas excludentes ou complementares.

Enunciado: Considere a seguinte estrutura lógica expressa em pseudocódigo:

```
Se (N1 = N2) Então
  Escreva (“São Iguais”)
Senão
  Se (N1 > N2) Então
    Escreva (“Maior: ”, N1)
  Senão
    Escreva (“Maior: ”, N2)
  Fim-Se
Fim-Se
```

Ao materializar este comportamento lógico diretamente em código Python, respeitando as boas práticas de otimização sintática da linguagem, qual estrutura representa fielmente o desvio aninhado acima?

A) O uso sequencial do comando switch/case integrado a operadores de conjunção.

B) A construção estruturada com as instruções if N1 == N2:, seguido de cláusula aninhada ou otimizada via elif N1 > N2: e o fechamento padrão em else:.

C) A implementação exclusiva de laços de repetição while controlados por interrupções condicionais do tipo break.

D) A aplicação direta do operador de permuta de referências N1, N2 = N2, N1.

E) A declaração de classes imutáveis decoradas com o rótulo @dataclass.

- **Gabarito:** B
- **Justificativa:** Em Python, o encadeamento de desvios condicionais mutuamente exclusivos baseados em testes lógicos sucessivos é implementado pela palavra-chave composto if, elif (redução sintática para *else if*) e else, mapeando perfeitamente os desvios aninhados de algoritmos tradicionais.

Questão 4: Estruturas de Repetição Indefinida e Critérios de Parada (Aula 3 e Aula 5)

Contexto: O desenvolvimento de loops infinitos controlados internamente por desvios lógicos representa uma arquitetura comum na automação de processos interativos, repetindo ações até que um gatilho explícito force a interrupção.

Enunciado: Analise o seguinte trecho de código Python estruturado para execução em um ambiente de *Notebook*:

```
while True:
  n1 = int(input("Digite o primeiro número: "))
  n2 = int(input("Digite o segundo número: "))

  if n1 == 0 and n2 == 0:
    break

  if n1 == n2:
    print("São Iguais")
  else:
    print(f"Maior: {max(n1, n2)}")
```

O comando de interrupção break desempenha um papel crítico na execução desse script. Esse papel corresponde a:

A) Reinicializar o escopo e o tempo de vida das variáveis n1 e n2 para que se tornem globais.

B) Provocar uma coerção de tipo implícita, forçando o interpretador a converter inteiros em strings.

C) Encerrar de forma abrupta e imediata a execução do laço de repetição while, desviando o fluxo do programa para a primeira instrução localizada logo após o bloco do loop.

D) Invocar um procedimento nativo do sistema para limpar a memória cache e otimizar o hardware.

E) Atuar como um operador relacional idêntico ao == para testar endereços de memória.

- **Gabarito:** C
- **Justificativa:** O comando break (correspondente ao "Interrompa" do pseudocódigo) serve exclusivamente para forçar a saída imediata de um bloco de repetição (while ou for), quebrando o fluxo circular e enviando o ponteiro de execução para a instrução subsequente ao loop.

Questão 5: Filosofia da Linguagem Python e Ambiência no Google Colab (Aula 5)

Contexto: Diferente de linguagens rígidas como C ou Java, Python foi projetado por Guido van Rossum focando na produtividade e na legibilidade. Seus princípios, descritos no *Zen do Python*, estabelecem que "bonito é melhor que feio", "explícito é melhor que implícito" e "simples é melhor que complexo". No laboratório acadêmico do Unilasalle, as práticas utilizam o Google Colab como IDE.

Enunciado: Sobre os fundamentos do Python e o funcionamento prático do ambiente do Google Colab, assinale a opção correta:

A) Python impõe tipagem estática e obrigatória, exigindo que o desenvolvedor declare explicitamente o tipo de cada variável antes de utilizá-la.

B) O Google Colab armazena os notebooks exclusivamente de forma volátil na memória RAM local da máquina do usuário, impedindo conexões e salvamentos automáticos no Google Drive.

C) Para escrever e testar códigos na prática no Google Colab, o código é digitado em células de código e executado ao pressionar as teclas Shift + Enter (ou clicando no botão de reprodução à esquerda), exibindo a saída logo abaixo.

D) O nome da linguagem foi escolhido por Guido devido à sua profunda obsessão zoológica por répteis de grande porte da família das pítons.

E) Python dispensa a utilização de regras de indentação, baseando a delimitação de seus blocos lógicos no uso obrigatório de chaves {} como em C.

- **Gabarito:** C
- **Justificativa:** O Google Colab funciona de forma baseada em células interativas integradas ao ecossistema Google Drive. O atalho de teclado padrão Shift + Enter compila/interpreta a célula atual em tempo real e renderiza os retornos lógicos imediatamente abaixo dela. Python usa tipagem dinâmica e adota indentação obrigatória para legibilidade em vez de chaves ou declarações rígidas de tipo.

Questão 6: Entrada de Dados Interativa e Coerção Primitiva de Tipos (Aula 5)

Contexto: Para interagir com usuários e receber dados digitados externamente por dispositivos de entrada, o interpretador Python disponibiliza universalmente a função embutida input().

Enunciado: Considere que um aluno do Unilasalle executou o seguinte script em um notebook do Colab:

```
nota_p1 = input("Digite a nota da P1: ")

nota_p2 = input("Digite a nota da P2: ")

media_final = (nota_p1 + nota_p2) / 2

print(f"Média: {media_final}")
```

Ao inserir os valores numéricos 8.0 e 10.0, o interpretador do Python interromperá o fluxo disparando uma exceção de tipo (*TypeError*). A causa técnica correta para este comportamento é:

A) A ausência de colchetes de indexação na atribuição matemática de media_final.

B) O fato de que a função input() captura dados exclusivamente em formato de texto (string), gerando um erro ao tentar aplicar operações de divisão sobre strings sem uma conversão explícita prévia (coerção) via float() ou int().

C) O caractere de f-string utilizado na função print() que impede cálculos retroativos na memória RAM.

D) Uma limitação específica da infraestrutura na nuvem do Google Colab que veda a divisão por números pares.

E) A redefinição inadequada de variáveis dinâmicas mutáveis que colidiram com o escopo global.

- **Gabarito:** B
- **Justificativa:** Por especificação nativa do Python, qualquer dado retornado pela função input() é tratado estritamente como string (str), mesmo que o texto consista em caracteres numéricos. Tentar realizar cálculos aritméticos (como divisão ou subtração) diretamente com strings resulta em falhas de tipos. É obrigatório envolver a entrada em funções de conversão, como float(input(...)).

Questão 7: Modularização e Taxonomia das Sub-rotinas (Aula 10)

Contexto: A modularização fundamenta-se no princípio de isolar blocos de código voltados a cumprir tarefas específicas. Seus benefícios principais englobam a manutenção ágil (facilidade para corrigir erros de forma isolada) e o reuso de código (escrever uma vez, usar em qualquer lugar).

Enunciado: A taxonomia de programação divide as sub-rotinas em dois tipos principais: **Procedimentos** e **Funções**. Assinale a alternativa que diferencia corretamente essas estruturas no contexto prático do Python.

A) Procedimentos operam de maneira autônoma na nuvem do Google Drive, enquanto funções rodam apenas na memória volátil local.

B) Procedimentos realizam uma série de instruções lógicas sem retornar um valor explícito para o ponto de chamada, ao passo que as Funções executam suas tarefas e devolvem obrigatoriamente um valor ao bloco chamador através da palavra-chave return.

C) Em Python, procedimentos são declarados pela palavra-chave procedure e as funções utilizam a sintaxe def.

D) Funções admitem apenas dados homogêneos atômicos como parâmetros de entrada, enquanto procedimentos processam vetores multivalorados.

E) Procedimentos aumentam significativamente o tamanho final dos arquivos de código fonte, enquanto funções operam limpando ponteiros.

- **Gabarito:** B
- **Justificativa:** Conforme detalhado no material, a diferença conceitual reside no destino da computação: procedimentos apenas executam uma tarefa imperativa, enquanto funções geram um resultado que é transmitido de volta para quem as invocou por meio da instrução de retorno return. Ambos os formatos utilizam a palavra-chave def para sua assinatura em Python.

Questão 8: Escopo Lógico e Modificação de Variáveis Globais (Aula 10)

Contexto: O escopo define as fronteiras de visibilidade de dados no ciclo de vida do software. Variáveis Locais existem apenas internamente no bloco da sub-rotina onde foram criadas, enquanto Variáveis Globais pertencem ao corpo principal do programa.

Enunciado: Analise o comportamento do código abaixo:

```
taxa_juros = 0.05

def atualizar_taxa():
    global taxa_juros
    taxa_juros = 0.08
    mensagem = "Taxa Atualizada"

atualizar_taxa()
print(taxa_juros)
```

Após a chamada de atualizar_taxa(), qual valor será exibido pelo print e o que acontece se o programador tentar imprimir a variável mensagem no escopo principal?

A) Será exibido o valor 0.05, e a variável mensagem estará acessível globalmente.

B) Será gerado um erro de compilação, pois o Python impede o uso simultâneo de variáveis locais e globais em uma mesma célula do Colab.

C) Será exibido o valor 0.08. Qualquer tentativa de ler a variável mensagem no escopo principal gerará um erro de nome (*NameError*), porque seu escopo é local e restrito ao bloco interno da função.

D) Será exibido o valor 0.13, decorrente da soma implícita de referências promovida pela instrução global.

E) Será exibido o texto "Taxa Atualizada", pois strings possuem precedência em alocações dinâmicas de memória RAM.

- **Gabarito:** C
- **Justificativa:** A instrução explicita global taxa_juros instrui o Python a direcionar as operações efetuadas dentro da sub-rotina diretamente à variável pertencente ao escopo principal do script, modificando seu valor de 0.05 para 0.08. Por outro lado, a variável mensagem possui escopo local, sendo desalocada e destruída da memória assim que a função encerra sua execução, tornando-se inacessível externamente.

Questão 9: Gerenciamento de Memória, Referências e Identidade de Objetos (Aula 12)

Contexto: Em termos de baixo nível, uma variável funciona como um rótulo que referencia uma posição ou endereço físico na memória RAM. O interpretador Python gerencia esse processo por meio de ponteiros automáticos. Para poupar recursos de hardware e RAM, o interpretador reaproveita alocações de objetos imutáveis equivalentes.

Enunciado: Considere a execução do seguinte script didático:

```
texto_a = "Analise de Algoritmos"

texto_b = texto_a


if id(texto_a) == id(texto_b):

  print("Sucesso: Apontam para a mesma memória!")

else:

  print("Falha: Endereços distintos detectados.")
```

Com base nas regras de gerenciamento de ponteiros e referências do Python, assinale a afirmativa correta sobre o comportamento do código acima:

A) O resultado impresso será "Falha: Endereços distintos detectados", porque o operador de igualdade = força a duplicação física da string inteira na RAM para preservar o backup.

B) O resultado impresso será "Sucesso: Apontam para a mesma memória!", porque o Python economiza RAM ao fazer com que texto_b torne-se um novo ponteiro apontando exatamente para o endereço de memória já ocupado por texto_a, evitando duplicações desnecessárias.

C) O script resultará em um erro em tempo de execução, dado que a função id() opera exclusivamente com tipos inteiros primitivos de bases atômicas.

D) O interpretador lançará um aviso de vazamento de memória devido à sobreposição de referências.

E) As duas variáveis serão promovidas automaticamente a sub-rotinas do tipo procedimento devido à equivalência.

- **Gabarito:** B
- **Justificativa:** Conforme demonstrado nos exemplos práticos de ponteiros em Python, realizar a atribuição de uma variável a outra não copia o conteúdo textual na memória; apenas copia o ponteiro (referência), fazendo com que ambas as variáveis compartilhem o mesmo endereço de memória, verificado pela igualdade de suas funções de identidade id().

Questão 10: Permuta de Referências em Linha Única (Aula 12)

Contexto: A inversão de valores armazenados em variáveis (operação de *swap*) tradicionalmente requer o uso de uma variável auxiliar temporária para evitar que a atribuição de um dado apague o outro acidentalmente na memória RAM.

Enunciado: Em Python, o programador consegue realizar a inversão de ponteiros de forma otimizada em uma única linha de instrução:

Python

```
val_x = 99

val_y = 55

val_x, val_y = val_y, val_x
```

Do ponto de vista da gestão interna de ponteiros promovida pela linguagem, o que acontece exatamente na linha em que é feita a atribuição simultânea?

A) O Python realiza uma operação aritmética implícita para calcular a diferença absoluta e realocar os bits.

B) Ocorre a troca instantânea das referências de memória: o ponteiro val_x passa a apontar para o endereço físico onde o valor 55 estava armazenado, enquanto o ponteiro val_y é redirecionado para o endereço de 99.

C) O interpretador Python interrompe o programa principal para disparar uma rotina recursiva de limpeza de hardware.

D) As duas variáveis transformam-se em dados multivalorados atômicos contendo matrizes idênticas.

E) A variável auxiliar temporária é alocada de forma oculta no BIOS da máquina.

- **Gabarito:** B
- **Justificativa:** Python avalia expressões avaliando integralmente o lado direito da igualdade primeiro (criando uma tupla temporária contendo as referências originais de memória) e depois desempacota-as na mesma ordem para o lado esquerdo. Isso troca a associação dos ponteiros de lugar sem a necessidade explícita de manipulação de código auxiliar extra.

Questão 11: Taxonomia Estrutural de Dados e Vetores Unidimensionais (Aula 13)

Contexto: De acordo com a taxonomia das variáveis apresentada em aula, as estruturas são categorizadas pela sua forma (Homogêneos ou Heterogêneos) e distribuição de valores (Atômicos ou Multivalorados).

Enunciado: Imagine que um sistema acadêmico armazene as notas finais de uma turma através de uma estrutura de dados indexada em Python:

Python

```
notas_alunos = [8.5, 7.0, 9.5]

notas_alunos.append(10.0)
```

Considerando que todos os elementos armazenados pertencem estritamente ao mesmo tipo primitivo decimal, esta coleção classifica-se na taxonomia estudada como um dado:

A) Heterogêneo atômico.

B) Homogêneo multivalorado.

C) Heterogêneo multivalorado estruturado.

D) Atômico composto imutável.

E) Procedimento matricial indexado.

- **Gabarito:** B
- **Justificativa:** Estruturas de dados formadas por múltiplos elementos pertencentes estritamente ao mesmo tipo de dado primitivo na memória (como uma lista contendo somente floats) enquadram-se na definição taxonômica de dados homogêneos multivalorados. O método .append() adiciona dinamicamente valores à extremidade final desse vetor.

Questão 12: Matrizes Bidimensionais e Indexação de Elementos (Aula 13)

Contexto: Matrizes são arranjos bidimensionais estruturados que organizam elementos sob a forma lógica de linhas e colunas. No Python, essas estruturas são nativamente representadas por meio do aninhamento de listas (listas de listas).

Enunciado: Um estudante do Unilasalle desenvolveu o código abaixo em seu ambiente laboratorial do Colab:

Python

```
matriz_dados = [[10, 20], [30, 40]]

nova_linha = [50, 60]

matriz_dados.append(nova_linha)

print(matriz_dados[2][1])
```

Após o processamento integral das linhas de instrução, qual valor será exibido no console e qual a configuração dimensional final adquirida pela matriz?

A) Exibirá o valor 40, e a matriz permanece com tamanho 2x2.

B) Exibirá o valor 50, e a dimensão da matriz altera-se para 2x3.

C) Exibirá o valor 60, e a matriz adquire a configuração final de 3 linhas e 2 colunas (3x2).

D) Ocorrerá uma exceção de índice estourado (*IndexError*), pois listas aninhadas não toleram expansões dinâmicas via append.

E) Exibirá o valor 10, considerando que a indexação em Python inicia-se em 1.

- **Gabarito:** C
- **Justificativa:** A matriz original possui duas linhas (índices 0 e 1). O método matriz_dados.append(nova_linha) insere o vetor [50, 60] na posição de índice 2,

tornando a matriz um arranjo de 3 linhas por 2 colunas (3x2). O acesso matriz_dados[2][1] pesquisa a linha de índice 2 e extrai o elemento da coluna de índice 1, que equivale ao número 60.

Questão 13: Registros e Estruturas Heterogêneas via Dataclasses (Aula 15)

Contexto: Diferente dos vetores homogêneos, um **Registro** (ou Estrutura) é um tipo de dado heterogêneo multivalorado concebido para reunir informações de naturezas e tipos diferentes sob uma mesma ficha cadastral unificada. Como o Python não possui a palavra-chave tradicional struct, adota-se a biblioteca padrão dataclasses para suprir esse molde conceitual.

Enunciado: Considere a definição abaixo estruturada para gerenciar o estoque do almoxarifado do campus do Unilasalle:

Python

```
from dataclasses import dataclass


@dataclass
class ItemAlmoxarifado:
  descricao: str
  codigo: int
  custo_unitario: float
```

Com base na aplicação do conceito de registros em Python, assinale a opção correta:

A) Todos os campos de uma classe decorada com @dataclass são obrigados a aceitar e converter dados estritamente em strings em tempo de execução.

B) Para instanciar adequadamente um registro a partir deste molde e imprimir seu campo descritivo, emprega-se a sintaxe item1 = ItemAlmoxarifado("Cadeira", 1001, 89.90) combinado ao acesso via notação de ponto: print(item1.descricao).

C) O uso de @dataclass converte a classe em um procedimento imperativo imutável que veda operações aritméticas posteriores.

D) A estrutura de dataclass restringe as variáveis criadas a operarem unicamente no escopo local de funções.

E) Registros são enquadrados como dados atômicos idênticos em virtude da ausência de loops internos.

- **Gabarito:** B
- **Justificativa:** Dataclasses geram métodos internos estruturados automaticamente (como o construtor), permitindo instanciar objetos passando argumentos para seus respectivos atributos. O acesso individual a cada campo armazenado no registro heterogêneo é efetuado por meio da notação de ponto (objeto.atributo).

Questão 14: Algoritmos de Varredura e Filtros em Coleções de Registros (Aula 15)

Contexto: A combinação de vetores (listas) com registros heterogêneos permite a criação de estruturas complexas na memória capazes de simular o comportamento de tabelas de bancos de dados relacionais.

Enunciado: Analise o código Python a seguir, desenvolvido para processamento de desempenho esportivo de uma equipe do Unilasalle:

```
from dataclasses import dataclass

@dataclass
class Jogador:
    nickname: str
    pontuacao: int
    nivel: int

time = [
    Jogador(nickname="GamerA", pontuacao=150, nivel=5),
    Jogador(nickname="GamerB", pontuacao=320, nivel=12),
    Jogador(nickname="GamerC", pontuacao=210, nivel=8)
]

acumulador = 0
for jg in time:
    if jg.nivel >= 8:
        acumulador += jg.pontuacao

print(acumulador)
```

Qual valor será exibido no console após o processamento completo do laço de repetição e da condicional?

A) 150

B) 320

C) 530

D) 680

E) 470

- **Gabarito:** C
- **Justificativa:** O loop for percorre cada instância de Jogador contida na lista time. A instrução condicional if jg.nivel >= 8 filtra apenas os jogadores cujo nível atende ao critério de corte. O GamerA (nível 5) é descartado. O GamerB (nível 12) é aceito, somando 320. O GamerC (nível 8) também é aceito, somando 210. O total acumulado corresponde a 320 + 210 = 530

  .

Questão 15: Manipulação de Strings, Métodos Prontos e Imutabilidade (Aula 16)

Contexto: Em Python, strings são sequências estruturadas de caracteres utilizadas para dados textuais. Elas possuem a propriedade da imutabilidade: uma vez criadas na memória, não podem ser alteradas; qualquer operação de modificação gera um objeto inteiramente novo. Métodos nativos como .split() e .join() são amplamente usados para o tratamento e limpeza de strings brutas ou arquivos padronizados (como CSV).

Enunciado: Considere a execução do seguinte código em uma célula do Google Colab:

Python

```
dados_brutos = "marcos,augusto,silva"

lista_nomes = dados_brutos.split(",")

resultado_final = " - ".join(lista_nomes)

print(resultado_final)
```

A saída impressa no console pela execução da variável resultado_final será correspondente a:

A) ['marcos', 'augusto', 'silva']

B) marcos - augusto - silva

C) marcos,augusto,silva -

D) Uma string vazia em decorrência da imutabilidade do objeto original.

E) Um erro de execução do tipo *AttributeError*, pois strings em Python não comportam métodos.

- **Gabarito:** B
- **Justificativa:** Conforme o material didático sobre processamento de strings, o método .split(",") atua fatiando o texto original com base no delimitador fornecido (a vírgula), retornando uma lista com os termos individuais. Posteriormente, o método " - ".join(lista_nomes) une os itens contidos nessa lista, intercalando-os com o caractere conector fornecido, gerando a string formatada final marcos - augusto - silva.